# EXP9

September 02, 2026

[ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

[ ]:
```
df = pd.read_csv("adult.csv")

# Data exploration
print("First Five Records\n")
print(df.head())

print("last Five Records\n")
print(df.tail())

print("\nDataset Information\n")
print(df.info())

print("\nDataset Description\n")
print(df.describe())

print("\nDataset Shape")
print(df.shape)
```

```
First Five Records

   age  workclass  fnlwgt     education  educational-num      marital-status  \
```

```
0   25    Private  226802          11th                7       Never-married
1   38    Private   89814       HS-grad                9  Married-civ-spouse
2   28  Local-gov  336951    Assoc-acdm               12  Married-civ-spouse
3   44    Private  160323  Some-college               10  Married-civ-spouse
4   18          ?  103497  Some-college               10       Never-married

          occupation relationship   race  gender  capital-gain  capital-loss  \
0  Machine-op-inspct    Own-child  Black    Male             0             0
1    Farming-fishing      Husband  White    Male             0             0
2    Protective-serv      Husband  White    Male             0             0
3  Machine-op-inspct      Husband  Black    Male          7688             0
4                  ?    Own-child  White  Female             0             0

   hours-per-week native-country income
0              40  United-States  <=50K
1              50  United-States  <=50K
2              40  United-States   >50K
3              40  United-States   >50K
4              30  United-States  <=50K
last Five Records

       age     workclass  fnlwgt   education  educational-num  \
48837   27       Private  257302  Assoc-acdm               12
48838   40       Private  154374     HS-grad                9
48839   58       Private  151910     HS-grad                9
48840   22       Private  201490     HS-grad                9
48841   52  Self-emp-inc  287927     HS-grad                9

           marital-status         occupation relationship   race  gender  \
48837  Married-civ-spouse       Tech-support         Wife  White  Female
48838  Married-civ-spouse  Machine-op-inspct      Husband  White    Male
48839             Widowed       Adm-clerical    Unmarried  White  Female
48840       Never-married       Adm-clerical    Own-child  White    Male
48841  Married-civ-spouse    Exec-managerial         Wife  White  Female

       capital-gain  capital-loss  hours-per-week native-country income
48837             0             0              38  United-States  <=50K
48838             0             0              40  United-States   >50K
48839             0             0              40  United-States  <=50K
48840             0             0              20  United-States  <=50K
48841         15024             0              40  United-States   >50K

Dataset Information

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48842 entries, 0 to 48841
Data columns (total 15 columns):
 #   Column           Non-Null Count  Dtype
```

```
---  ------           --------------  -----
 0   age              48842 non-null  int64
 1   workclass        48842 non-null  object
 2   fnlwgt           48842 non-null  int64
 3   education        48842 non-null  object
 4   educational-num  48842 non-null  int64
 5   marital-status   48842 non-null  object
 6   occupation       48842 non-null  object
 7   relationship     48842 non-null  object
 8   race             48842 non-null  object
 9   gender           48842 non-null  object
 10  capital-gain     48842 non-null  int64
 11  capital-loss     48842 non-null  int64
 12  hours-per-week   48842 non-null  int64
 13  native-country   48842 non-null  object
 14  income           48842 non-null  object
dtypes: int64(6), object(9)
memory usage: 5.6+ MB
None

Dataset Description

                age        fnlwgt  educational-num  capital-gain  \
count  48842.000000  4.884200e+04     48842.000000  48842.000000
mean      38.643585  1.896641e+05        10.078089   1079.067626
std       13.710510  1.056040e+05         2.570973   7452.019058
min       17.000000  1.228500e+04         1.000000      0.000000
25%       28.000000  1.175505e+05         9.000000      0.000000
50%       37.000000  1.781445e+05        10.000000      0.000000
75%       48.000000  2.376420e+05        12.000000      0.000000
max       90.000000  1.490400e+06        16.000000  99999.000000

       capital-loss  hours-per-week
count  48842.000000    48842.000000
mean      87.502314       40.422382
std      403.004552       12.391444
min        0.000000        1.000000
25%        0.000000       40.000000
50%        0.000000       40.000000
75%        0.000000       45.000000
max     4356.000000       99.000000

Dataset Shape
(48842, 15)
```

[ ]:
```
df.columns = df.columns.str.strip()
```

```
df.replace("?", np.nan, inplace=True)

for column in df.select_dtypes(include="object").columns:
    df[column] = df[column].str.strip()

print("\nMissing Values")
print(df.isnull().sum())

df.dropna(inplace=True)

df.drop_duplicates(inplace=True)

df.reset_index(drop=True, inplace=True)

print("\nDataset Shape After Removing Missing Values:")
print(df.shape)
```

```
Missing Values
age                  0
workclass         2799
fnlwgt               0
education            0
educational-num      0
marital-status       0
occupation        2809
relationship         0
race                 0
gender               0
capital-gain         0
capital-loss         0
hours-per-week       0
native-country     857
income               0
dtype: int64

Dataset Shape After Removing Missing Values:
(45175, 15)
```

[ ]:
```
encoder = LabelEncoder()

for column in df.columns:
    if df[column].dtype == "object":
        df[column] = encoder.fit_transform(df[column])

print("\nColumn Names:")
print(df.columns)
```

```
X = df.drop("income", axis=1)
y = df["income"]
```

```
Column Names:
Index(['age', 'workclass', 'fnlwgt', 'education', 'educational-num',
       'marital-status', 'occupation', 'relationship', 'race', 'gender',
       'capital-gain', 'capital-loss', 'hours-per-week', 'native-country',
       'income'],
      dtype='object')
```

[ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (36140, 14)
Testing Data Shape: (9035, 14)
```

[ ]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

[ ]:
```
logistic_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)
logistic_model.fit(X_train_scaled, y_train)

y_pred_lr = logistic_model.predict(X_test_scaled)
```

[ ]:
```
decision_tree_model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=5,
    random_state=42
)
decision_tree_model.fit(X_train, y_train)

y_pred_dt = decision_tree_model.predict(X_test)
```

[ ]:
```
def evaluate_model(y_test, y_pred, model_name):

    accuracy = accuracy_score(y_test, y_pred)

    precision = precision_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    recall = recall_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    f1 = f1_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    print(model_name)
    print("Accuracy :", accuracy)
    print("Precision:", precision)
    print("Recall   :", recall)
    print("F1 Score :", f1)

    print("\nClassification Report:")
    print(classification_report(
        y_test,
        y_pred,
        zero_division=0
    ))

    return accuracy, precision, recall, f1

lr_results = evaluate_model(
    y_test,
    y_pred_lr,
    "LOGISTIC REGRESSION"
)

dt_results = evaluate_model(
```

```
    y_test,
    y_pred_dt,
    "DECISION TREE"
)

comparison = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Decision Tree"
    ],
    "Accuracy": [
        lr_results[0],
        dt_results[0]
    ],
    "Precision": [
        lr_results[1],
        dt_results[1]
    ],
    "Recall": [
        lr_results[2],
        dt_results[2]
    ],
    "F1 Score": [
        lr_results[3],
        dt_results[3]
    ]
})

print("MODEL COMPARISON")
print(comparison)
```

```
LOGISTIC REGRESSION
Accuracy : 0.8148312119535142
Precision: 0.8024155622955101
Recall   : 0.8148312119535142
F1 Score : 0.7995801900619552

Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.94      0.88      6795
           1       0.70      0.45      0.54      2240

    accuracy                           0.81      9035
   macro avg       0.77      0.69      0.71      9035
weighted avg       0.80      0.81      0.80      9035

DECISION TREE
```

```
Accuracy : 0.8426120641947981
Precision: 0.8366133303082204
Recall   : 0.8426120641947981
F1 Score : 0.8296281113232561

Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.96      0.90      6795
           1       0.79      0.50      0.61      2240

    accuracy                           0.84      9035
   macro avg       0.82      0.73      0.76      9035
weighted avg       0.84      0.84      0.83      9035

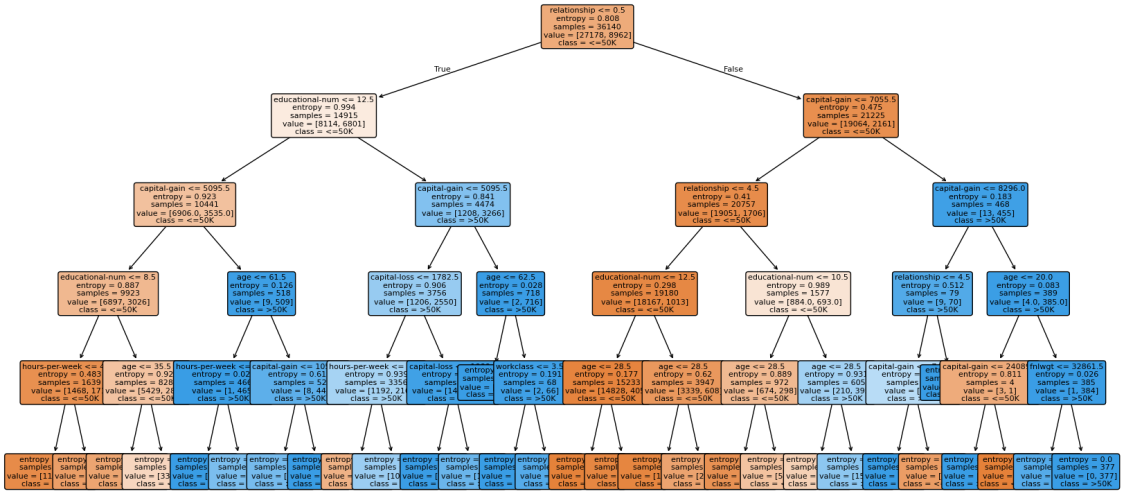
MODEL COMPARISON
                 Model  Accuracy  Precision    Recall  F1 Score
0  Logistic Regression  0.814831   0.802416  0.814831  0.799580
1        Decision Tree  0.842612   0.836613  0.842612  0.829628
```

[ ]:
```python
plt.figure(figsize=(20, 10))

plot_tree(
    decision_tree_model,
    feature_names=X.columns,
    class_names=["<=50K", ">50K"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Adult Income Prediction")
plt.show()
```

Decision Tree for Adult Income Prediction



# EXP10

September 09, 2026

[ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix
```

[ ]:
```
df = pd.read_csv(r"/content/Iris.csv")
print(df.head())
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

[ ]:
```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
None
```

[ ]:
```
print(df.shape)
print(df.describe())
```

```
(150, 6)
```

| | Id | SepalLengthCm | SepalWidthCm | PetalLengthCm | PetalWidthCm |
|---|---|---|---|---|---|
| count | 150.000000 | 150.000000 | 150.000000 | 150.000000 | 150.000000 |
| mean | 75.500000 | 5.843333 | 3.054000 | 3.758667 | 1.198667 |
| std | 43.445368 | 0.828066 | 0.433594 | 1.764420 | 0.763161 |
| min | 1.000000 | 4.300000 | 2.000000 | 1.000000 | 0.100000 |
| 25% | 38.250000 | 5.100000 | 2.800000 | 1.600000 | 0.300000 |
| 50% | 75.500000 | 5.800000 | 3.000000 | 4.350000 | 1.300000 |
| 75% | 112.750000 | 6.400000 | 3.300000 | 5.100000 | 1.800000 |
| max | 150.000000 | 7.900000 | 4.400000 | 6.900000 | 2.500000 |

[ ]:
```
if "Id" in df.columns:
  df = df.drop("Id", axis = 1)

df = df[df["Species"] != "Iris-virginica"].copy()
encoder = LabelEncoder()
df["species"] = encoder.fit_transform(df["Species"])

X = df[["SepalLengthCm", "SepalWidthCm"]].values
y = df["species"].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,␣
 ↪random_state = 42, stratify = y)

scaler = StandardScaler()
X = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[ ]:
```
model = SVC(kernel = "linear", C = 1)
model.fit(X_train, y_train)

pred = model.predict(X_test)
```

[ ]:
```
print("Accuracy: ",accuracy_score(y_test, pred))
print("\nClassification Report:")
print(classification_report(y_test, pred, zero_division=0))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, pred))

X_all = np.vstack((X_train, X_test))
y_all = np.concatenate((y_train, y_test))

x_min, x_max = X_all[:, 0].min() - 1, X_all[:, 0].max() + 1
y_min, y_max = X_all[:, 1].min() - 1, X_all[:, 1].max() + 1

xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max, 0.
 ↪02))
```

```
Z = model.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

```
Accuracy:  0.5

Classification Report:
              precision    recall  f1-score   support

           0       0.50      1.00      0.67        10
           1       0.00      0.00      0.00        10

    accuracy                           0.50        20
   macro avg       0.25      0.50      0.33        20
weighted avg       0.25      0.50      0.33        20


Confusion Matrix:
[[10  0]
 [10  0]]
```

# EXP11

Septemeber 23, 2026

## 1 Multilayer Feed-Forward Network (MLP) on Wine Quality Dataset

[ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report,␣
↪confusion_matrix
```

[ ]:
```
# Load the Wine Quality dataset from local storage path
df = pd.read_csv('/content/winequality-red.csv', sep=';')

# Display metadata and statistical descriptions
print("First 5 records:")
display(df.head())

print("\nDataset Information:")
df.info()

print("\nStatistical Summary:")
display(df.describe())

print(f"\nDataset Dimensions: {df.shape}")

# Check for missing values
print("\nMissing values per column:")
print(df.isnull().sum())
```

```
First 5 records:
   fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0            7.4              0.70         0.00             1.9      0.076
1            7.8              0.88         0.00             2.6      0.098
```

```
2            7.8              0.76         0.04             2.3      0.092
3           11.2              0.28         0.56             1.9      0.075
4            7.4              0.70         0.00             1.9      0.076

   free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                 11.0                  34.0   0.9978  3.51       0.56
1                 25.0                  67.0   0.9968  3.20       0.68
2                 15.0                  54.0   0.9970  3.26       0.65
3                 17.0                  60.0   0.9980  3.16       0.58
4                 11.0                  34.0   0.9978  3.51       0.56

   alcohol  quality
0      9.4        5
1      9.8        5
2      9.8        5
3      9.8        6
4      9.4        5


Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         1599 non-null   float64
 1   volatile acidity      1599 non-null   float64
 2   citric acid           1599 non-null   float64
 3   residual sugar        1599 non-null   float64
 4   chlorides             1599 non-null   float64
 5   free sulfur dioxide   1599 non-null   float64
 6   total sulfur dioxide  1599 non-null   float64
 7   density               1599 non-null   float64
 8   pH                    1599 non-null   float64
 9   sulphates             1599 non-null   float64
 10  alcohol               1599 non-null   float64
 11  quality               1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB

Statistical Summary:

       fixed acidity  volatile acidity  citric acid  residual sugar  \
count    1599.000000       1599.000000  1599.000000     1599.000000
mean        8.319637          0.527821     0.270976        2.538806
std         1.741096          0.179060     0.194801        1.409928
min         4.600000          0.120000     0.000000        0.900000
25%         7.100000          0.390000     0.090000        1.900000
50%         7.900000          0.520000     0.260000        2.200000
```

```
75%          9.200000          0.640000     0.420000        2.600000
max         15.900000          1.580000     1.000000       15.500000

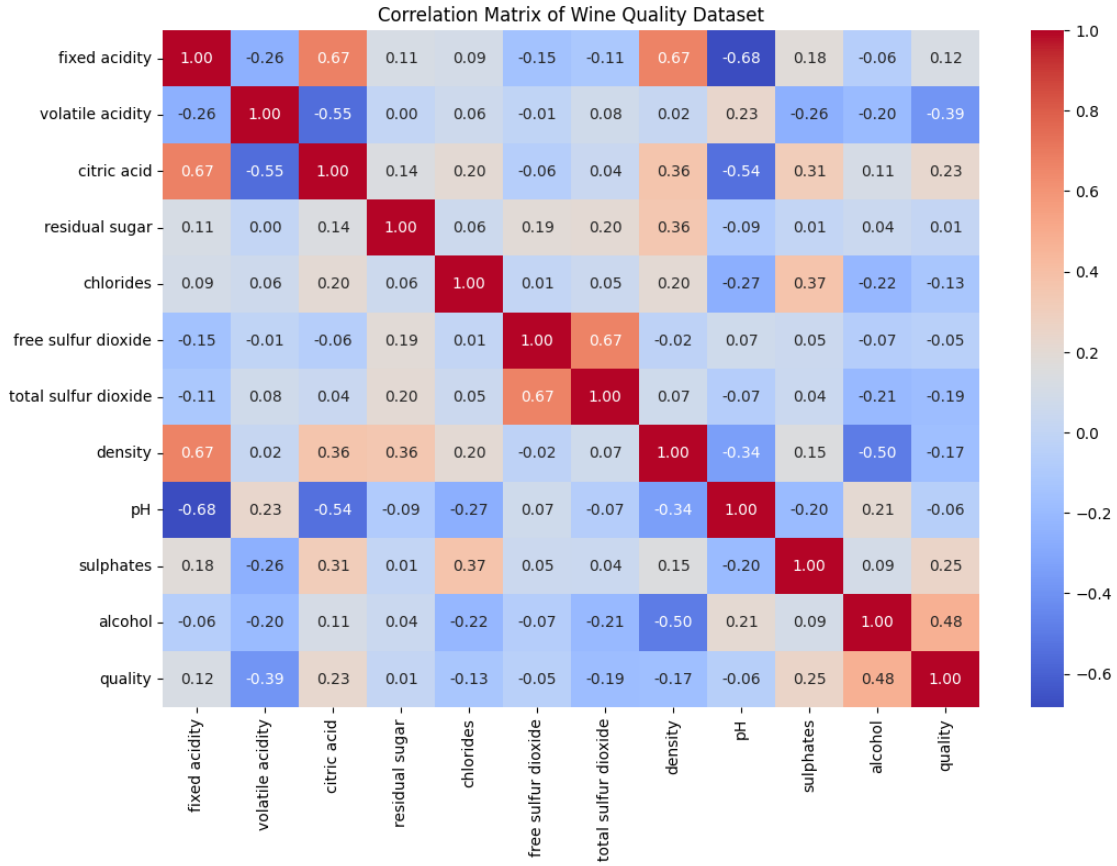
         chlorides  free sulfur dioxide  total sulfur dioxide      density  \
count  1599.000000          1599.000000           1599.000000  1599.000000
mean      0.087467            15.874922             46.467792     0.996747
std       0.047065            10.460157             32.895324     0.001887
min       0.012000             1.000000              6.000000     0.990070
25%       0.070000             7.000000             22.000000     0.995600
50%       0.079000            14.000000             38.000000     0.996750
75%       0.090000            21.000000             62.000000     0.997835
max       0.611000            72.000000            289.000000     1.003690

                pH    sulphates      alcohol      quality
count  1599.000000  1599.000000  1599.000000  1599.000000
mean      3.311113     0.658149    10.422983     5.636023
std       0.154386     0.169507     1.065668     0.807569
min       2.740000     0.330000     8.400000     3.000000
25%       3.210000     0.550000     9.500000     5.000000
50%       3.310000     0.620000    10.200000     6.000000
75%       3.400000     0.730000    11.100000     6.000000
max       4.010000     2.000000    14.900000     8.000000


Dataset Dimensions: (1599, 12)

Missing values per column:
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
dtype: int64
```

[ ]:
```
plt.figure(figsize=(12, 8))
correlation_matrix = df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Wine Quality Dataset')
plt.show()
```




[ ]:
```
# Separate Features (X) and Target Variable (y)
X = df.drop(columns=['quality'])
y = df['quality']

print(f"X dimensions: {X.shape}")
print(f"y dimensions: {y.shape}")

# Split the dataset (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

# Normalize the dataset using StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Data preprocessed and scaled successfully.")
```

```
X dimensions: (1599, 11)
y dimensions: (1599,)
```

```
Data preprocessed and scaled successfully.
```

[ ]:
```python
# Define and train the first MLP Architecture
mlp1 = MLPClassifier(hidden_layer_sizes=(50,), max_iter=500, random_state=42)
mlp1.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred1 = mlp1.predict(X_test_scaled)
accuracy1 = accuracy_score(y_test, y_pred1)

print(f"Architecture 1 (50,) Accuracy: {accuracy1:.4f}")
print("\nClassification Report for Architecture 1:")
print(classification_report(y_test, y_pred1, zero_division=0))
```

```
Architecture 1 (50,) Accuracy: 0.5969

Classification Report for Architecture 1:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00        10
           5       0.63      0.72      0.67       130
           6       0.59      0.60      0.60       132
           7       0.55      0.43      0.48        42
           8       0.00      0.00      0.00         5

    accuracy                           0.60       320
   macro avg       0.29      0.29      0.29       320
weighted avg       0.57      0.60      0.58       320


/usr/local/lib/python3.13/dist-
packages/sklearn/neural_network/_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and
the optimization hasn't converged yet.
  warnings.warn(
```

[ ]:
```python
# Define and train the second MLP Architecture
mlp2 = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500,␣
 ↪random_state=42)
mlp2.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred2 = mlp2.predict(X_test_scaled)
accuracy2 = accuracy_score(y_test, y_pred2)

print(f"Architecture 2 (100, 50) Accuracy: {accuracy2:.4f}")
print("\nClassification Report for Architecture 2:")
```

```
print(classification_report(y_test, y_pred2, zero_division=0))
```

```
Architecture 2 (100, 50) Accuracy: 0.6406

Classification Report for Architecture 2:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00        10
           5       0.69      0.73      0.71       130
           6       0.63      0.65      0.64       132
           7       0.57      0.57      0.57        42
           8       0.00      0.00      0.00         5

    accuracy                           0.64       320
   macro avg       0.32      0.33      0.32       320
weighted avg       0.62      0.64      0.63       320


/usr/local/lib/python3.13/dist-
packages/sklearn/neural_network/_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and
the optimization hasn't converged yet.
  warnings.warn(
```

[ ]:
```
# Define and train the third MLP Architecture
mlp3 = MLPClassifier(hidden_layer_sizes=(128, 64, 32), max_iter=500,␣
 ↪random_state=42)
mlp3.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred3 = mlp3.predict(X_test_scaled)
accuracy3 = accuracy_score(y_test, y_pred3)

print(f"Architecture 3 (128, 64, 32) Accuracy: {accuracy3:.4f}")
print("\nClassification Report for Architecture 3:")
print(classification_report(y_test, y_pred3, zero_division=0))
```

```
Architecture 3 (128, 64, 32) Accuracy: 0.6438

Classification Report for Architecture 3:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.43      0.30      0.35        10
           5       0.69      0.74      0.71       130
           6       0.63      0.61      0.62       132
           7       0.60      0.64      0.62        42
           8       0.00      0.00      0.00         5
```

| | | | | |
|---|---|---|---|---|
| accuracy | | | 0.64 | 320 |
| macro avg | 0.39 | 0.38 | 0.38 | 320 |
| weighted avg | 0.63 | 0.64 | 0.64 | 320 |

# EXP12

September 23, 2026

[1]:
```
import kagglehub
path = kagglehub.dataset_download("zalando-research/fashionmnist")
```

```
Using Colab cache for faster access to the 'fashionmnist' dataset.
```

[2]:
```
import pandas as pd
import numpy as np
import os

# Locate the CSV files in the downloaded directory
csv_files = []
for root, dirs, files in os.walk(path):
    for file in files:
        if file.endswith('.csv'):
            csv_files.append(os.path.join(root, file))

print("Found CSV files:", csv_files)

# Fallback to keras datasets if no CSV is found, else load CSVs
if len(csv_files) >= 2:
    train_path = [f for f in csv_files if 'train' in f.lower()][0]
    test_path = [f for f in csv_files if 'test' in f.lower()][0]

    train_df = pd.read_csv(train_path)
    test_df = pd.read_csv(test_path)

    print("\n--- Train Dataset Info ---")
    print(train_df.info())
    print("\n--- Train Dataset Statistical Summary ---")
    print(train_df.describe())

    y_train_raw = train_df.iloc[:, 0].values
    x_train_raw = train_df.iloc[:, 1:].values
    y_test_raw = test_df.iloc[:, 0].values
    x_test_raw = test_df.iloc[:, 1:].values
else:
    print("No CSVs found in path. Loading from Keras as fallback with␣
 ↪equivalent structure.")
```

```
    from tensorflow.keras.datasets import fashion_mnist
    (x_train_3d, y_train_raw), (x_test_3d, y_test_raw) = fashion_mnist.
↪load_data()
    x_train_raw = x_train_3d.reshape(-1, 784)
    x_test_raw = x_test_3d.reshape(-1, 784)
```

```
Found CSV files: ['/kaggle/input/fashionmnist/fashion-mnist_test.csv',
'/kaggle/input/fashionmnist/fashion-mnist_train.csv']

--- Train Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60000 entries, 0 to 59999
Columns: 785 entries, label to pixel784
dtypes: int64(785)
memory usage: 359.3 MB
None

--- Train Dataset Statistical Summary ---
              label        pixel1        pixel2        pixel3        pixel4  \
count  60000.000000  60000.000000  60000.000000  60000.000000  60000.000000
mean       4.500000      0.000900      0.006150      0.035333      0.101933
std        2.872305      0.094689      0.271011      1.222324      2.452871
min        0.000000      0.000000      0.000000      0.000000      0.000000
25%        2.000000      0.000000      0.000000      0.000000      0.000000
50%        4.500000      0.000000      0.000000      0.000000      0.000000
75%        7.000000      0.000000      0.000000      0.000000      0.000000
max        9.000000     16.000000     36.000000    226.000000    164.000000

             pixel5        pixel6        pixel7        pixel8        pixel9  \
count  60000.000000  60000.000000  60000.000000  60000.000000  60000.000000
mean       0.247967      0.411467      0.805767      2.198283      5.682000
std        4.306912      5.836188      8.215169     14.093378     23.819481
min        0.000000      0.000000      0.000000      0.000000      0.000000
25%        0.000000      0.000000      0.000000      0.000000      0.000000
50%        0.000000      0.000000      0.000000      0.000000      0.000000
75%        0.000000      0.000000      0.000000      0.000000      0.000000
max      227.000000    230.000000    224.000000    255.000000    254.000000

       …      pixel775      pixel776      pixel777      pixel778  \
count  …  60000.000000  60000.000000  60000.000000  60000.000000
mean   …     34.625400     23.300683     16.588267     17.869433
std    …     57.545242     48.854427     41.979611     43.966032
min    …      0.000000      0.000000      0.000000      0.000000
25%    …      0.000000      0.000000      0.000000      0.000000
50%    …      0.000000      0.000000      0.000000      0.000000
75%    …     58.000000      9.000000      0.000000      0.000000
max    …    255.000000    255.000000    255.000000    255.000000
```

```
           pixel779      pixel780      pixel781      pixel782      pixel783  \
count  60000.000000  60000.000000  60000.000000  60000.000000  60000.000000
mean      22.814817     17.911483      8.520633      2.753300      0.855517
std       51.830477     45.149388     29.614859     17.397652      9.356960
min        0.000000      0.000000      0.000000      0.000000      0.000000
25%        0.000000      0.000000      0.000000      0.000000      0.000000
50%        0.000000      0.000000      0.000000      0.000000      0.000000
75%        0.000000      0.000000      0.000000      0.000000      0.000000
max      255.000000    255.000000    255.000000    255.000000    255.000000

           pixel784
count  60000.00000
mean       0.07025
std        2.12587
min        0.00000
25%        0.00000
50%        0.00000
75%        0.00000
max      170.00000

[8 rows x 785 columns]
```

[3]:
```python
from tensorflow.keras.utils import to_categorical

# Normalize features
X_train = x_train_raw / 255.0
X_test = x_test_raw / 255.0

# One-hot encode targets
Y_train = to_categorical(y_train_raw, 10)
Y_test = to_categorical(y_test_raw, 10)

print(f"X_train shape: {X_train.shape}, Y_train shape: {Y_train.shape}")
print(f"X_test shape: {X_test.shape}, Y_test shape: {Y_test.shape}")
```

```
X_train shape: (60000, 784), Y_train shape: (60000, 10)
X_test shape: (10000, 784), Y_test shape: (10000, 10)
```

[4]:
```python
import time
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

results = {}

# Define activations to compare
activations = ['sigmoid', 'relu', 'tanh']
```

```
for act in activations:
    print(f"\nTraining network with {act.upper()} activation...")

    # Design the network
    model = Sequential([
        Dense(128, activation=act, input_shape=(784,)),
        Dense(64, activation=act),
        Dense(10, activation='softmax')
    ])

    # Compile
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    # Train & record time
    start_time = time.time()
    history = model.fit(X_train, Y_train, epochs=8, batch_size=64,␣
 ↪validation_split=0.1, verbose=1)
    end_time = time.time()

    training_time = end_time - start_time

    # Evaluate
    loss, accuracy = model.evaluate(X_test, Y_test, verbose=0)

    results[act] = {
        'history': history.history,
        'training_time': training_time,
        'accuracy': accuracy
    }
    print(f"{act.upper()} - Training Time: {training_time:.2f}s, Test Accuracy:␣
 ↪{accuracy:.4f}")
```

```
Training network with SIGMOID activation…

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/8
844/844             5s 4ms/step -
accuracy: 0.7661 - loss: 0.7343 - val_accuracy: 0.8267 - val_loss: 0.4859
Epoch 2/8
844/844             5s 6ms/step -
accuracy: 0.8473 - loss: 0.4260 - val_accuracy: 0.8508 - val_loss: 0.4209
```

```
Epoch 3/8
844/844             4s 4ms/step -
accuracy: 0.8643 - loss: 0.3782 - val_accuracy: 0.8612 - val_loss: 0.3839
Epoch 4/8
844/844             3s 4ms/step -
accuracy: 0.8732 - loss: 0.3511 - val_accuracy: 0.8685 - val_loss: 0.3635
Epoch 5/8
844/844             4s 5ms/step -
accuracy: 0.8793 - loss: 0.3302 - val_accuracy: 0.8713 - val_loss: 0.3604
Epoch 6/8
844/844             4s 4ms/step -
accuracy: 0.8846 - loss: 0.3178 - val_accuracy: 0.8742 - val_loss: 0.3501
Epoch 7/8
844/844             4s 4ms/step -
accuracy: 0.8890 - loss: 0.3049 - val_accuracy: 0.8778 - val_loss: 0.3380
Epoch 8/8
844/844             4s 4ms/step -
accuracy: 0.8919 - loss: 0.2940 - val_accuracy: 0.8847 - val_loss: 0.3259
SIGMOID - Training Time: 32.62s, Test Accuracy: 0.8806

Training network with RELU activation…
Epoch 1/8
844/844             5s 4ms/step -
accuracy: 0.8128 - loss: 0.5267 - val_accuracy: 0.8418 - val_loss: 0.4377
Epoch 2/8
844/844             3s 4ms/step -
accuracy: 0.8596 - loss: 0.3872 - val_accuracy: 0.8538 - val_loss: 0.3983
Epoch 3/8
844/844             4s 5ms/step -
accuracy: 0.8747 - loss: 0.3460 - val_accuracy: 0.8567 - val_loss: 0.3919
Epoch 4/8
844/844             4s 5ms/step -
accuracy: 0.8832 - loss: 0.3201 - val_accuracy: 0.8702 - val_loss: 0.3577
Epoch 5/8
844/844             4s 4ms/step -
accuracy: 0.8887 - loss: 0.3038 - val_accuracy: 0.8742 - val_loss: 0.3484
Epoch 6/8
844/844             4s 5ms/step -
accuracy: 0.8926 - loss: 0.2889 - val_accuracy: 0.8707 - val_loss: 0.3594
Epoch 7/8
844/844             5s 5ms/step -
accuracy: 0.8985 - loss: 0.2741 - val_accuracy: 0.8775 - val_loss: 0.3402
Epoch 8/8
844/844             3s 4ms/step -
accuracy: 0.9009 - loss: 0.2645 - val_accuracy: 0.8845 - val_loss: 0.3298
RELU - Training Time: 32.19s, Test Accuracy: 0.8860

Training network with TANH activation…
```

```
Epoch 1/8
844/844             6s 7ms/step -
accuracy: 0.8244 - loss: 0.4963 - val_accuracy: 0.8467 - val_loss: 0.4299
Epoch 2/8
844/844             4s 4ms/step -
accuracy: 0.8633 - loss: 0.3760 - val_accuracy: 0.8613 - val_loss: 0.3827
Epoch 3/8
844/844             3s 4ms/step -
accuracy: 0.8757 - loss: 0.3376 - val_accuracy: 0.8695 - val_loss: 0.3594
Epoch 4/8
844/844             7s 6ms/step -
accuracy: 0.8832 - loss: 0.3194 - val_accuracy: 0.8773 - val_loss: 0.3401
Epoch 5/8
844/844             3s 4ms/step -
accuracy: 0.8896 - loss: 0.2993 - val_accuracy: 0.8712 - val_loss: 0.3466
Epoch 6/8
844/844             3s 4ms/step -
accuracy: 0.8939 - loss: 0.2868 - val_accuracy: 0.8758 - val_loss: 0.3348
Epoch 7/8
844/844             4s 4ms/step -
accuracy: 0.9002 - loss: 0.2716 - val_accuracy: 0.8752 - val_loss: 0.3435
Epoch 8/8
844/844             4s 5ms/step -
accuracy: 0.9033 - loss: 0.2611 - val_accuracy: 0.8852 - val_loss: 0.3103
TANH - Training Time: 35.16s, Test Accuracy: 0.8868
```

[5]:
```python
import matplotlib.pyplot as plt

# Create comparison DataFrame
comparison_df = pd.DataFrame({
    'Activation Function': [act.upper() for act in results.keys()],
    'Training Time (seconds)': [results[act]['training_time'] for act in␣
 ↪results.keys()],
    'Classification Accuracy': [results[act]['accuracy'] for act in results.
 ↪keys()]
})

display(comparison_df)

# Plotting Classification Accuracy Comparison
plt.figure(figsize=(12, 5))

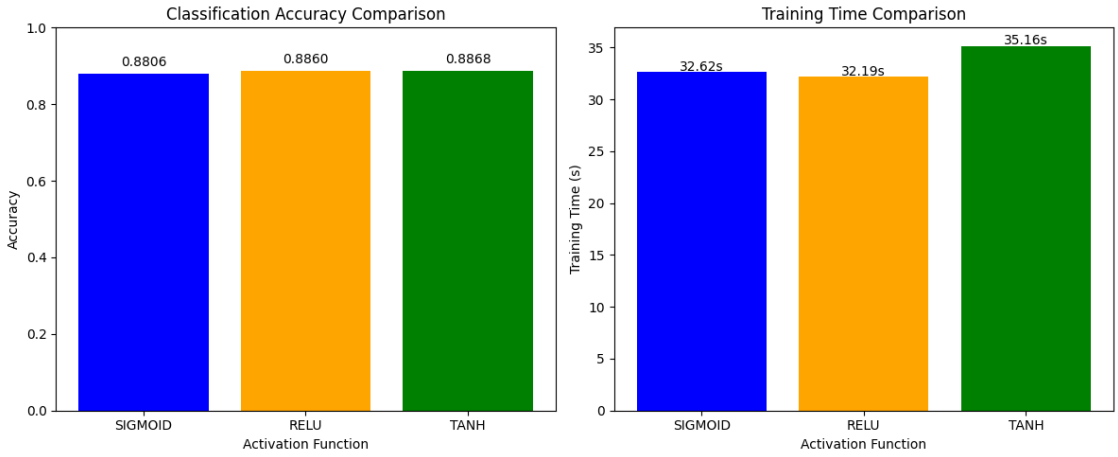
plt.subplot(1, 2, 1)
plt.bar(comparison_df['Activation Function'], comparison_df['Classification␣
 ↪Accuracy'], color=['blue', 'orange', 'green'])
plt.title('Classification Accuracy Comparison')
plt.xlabel('Activation Function')
```

```
plt.ylabel('Accuracy')
plt.ylim(0, 1.0)
for i, val in enumerate(comparison_df['Classification Accuracy']):
    plt.text(i, val + 0.02, f"{val:.4f}", ha='center')

# Plotting Training Time Comparison
plt.subplot(1, 2, 2)
plt.bar(comparison_df['Activation Function'], comparison_df['Training Time␣
 ↪(seconds)'], color=['blue', 'orange', 'green'])
plt.title('Training Time Comparison')
plt.xlabel('Activation Function')
plt.ylabel('Training Time (s)')
for i, val in enumerate(comparison_df['Training Time (seconds)']):
    plt.text(i, val + 0.1, f"{val:.2f}s", ha='center')

plt.tight_layout()
plt.show()
```

| | Activation Function | Training Time (seconds) | Classification Accuracy |
|---|---|---|---|
| 0 | SIGMOID | 32.615285 | 0.8806 |
| 1 | RELU | 32.185057 | 0.8860 |
| 2 | TANH | 35.155779 | 0.8868 |



# EXP13

October 09, 2026

[ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

[ ]:
```
digits = load_digits()
df = pd.DataFrame(digits.data)
df['Target'] = digits.target
print(df.head())
```

```
     0    1    2     3     4     5  …    59    60    61   62   63  Target
0  0.0  0.0  5.0  13.0   9.0   1.0  …  13.0  10.0   0.0  0.0  0.0       0
1  0.0  0.0  0.0  12.0  13.0   5.0  …  11.0  16.0  10.0  0.0  0.0       1
2  0.0  0.0  0.0   4.0  15.0  12.0  …   3.0  11.0  16.0  9.0  0.0       2
3  0.0  0.0  7.0  15.0  13.0   1.0  …  13.0  13.0   9.0  0.0  0.0       3
4  0.0  0.0  0.0   1.0  11.0   0.0  …   2.0  16.0   4.0  0.0  0.0       4

[5 rows x 65 columns]
```

[ ]:
```
X = digits.data
y = digits.target

print("Shape of dataset:", X.shape)
print("Number of classes:", len(np.unique(y)))
print("Classes:", np.unique(y))
```

```
Shape of dataset: (1797, 64)
Number of classes: 10
Classes: [0 1 2 3 4 5 6 7 8 9]
```

[ ]:
```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print("Original shape:", X.shape)
print("Scaled shape:", X_scaled.shape)
```

```
Original shape: (1797, 64)
Scaled shape: (1797, 64)
```

[ ]:
```python
inertia = []
silhouette_scores = []

k_values = range(2, 16)

for k in k_values:


    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )


    labels = kmeans.fit_predict(X_scaled)


    inertia.append(kmeans.inertia_)


    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

print("Experiment completed.")
```

```
Experiment completed.
```

[ ]:
```python
results = pd.DataFrame({
    "K": list(k_values),
    "Inertia": inertia,
    "Silhouette Score": silhouette_scores
})

print(results)
```

```
     K        Inertia  Silhouette Score
0    2  100423.843034          0.105638
1    3   93842.482749          0.105287
2    4   89213.968157          0.095386
3    5   85096.054668          0.102298
4    6   81678.466699          0.098789
5    7   77749.327896          0.113494
6    8   74854.159172          0.126590
7    9   72029.725334          0.135985
8   10   69813.559137          0.139377
```

| | | | |
|---|---|---|---|
| 9 | 11 | 67261.847461 | 0.153194 |
| 10 | 12 | 65031.723728 | 0.154325 |
| 11 | 13 | 62763.180754 | 0.156527 |
| 12 | 14 | 61963.418696 | 0.156195 |
| 13 | 15 | 59534.498937 | 0.158307 |

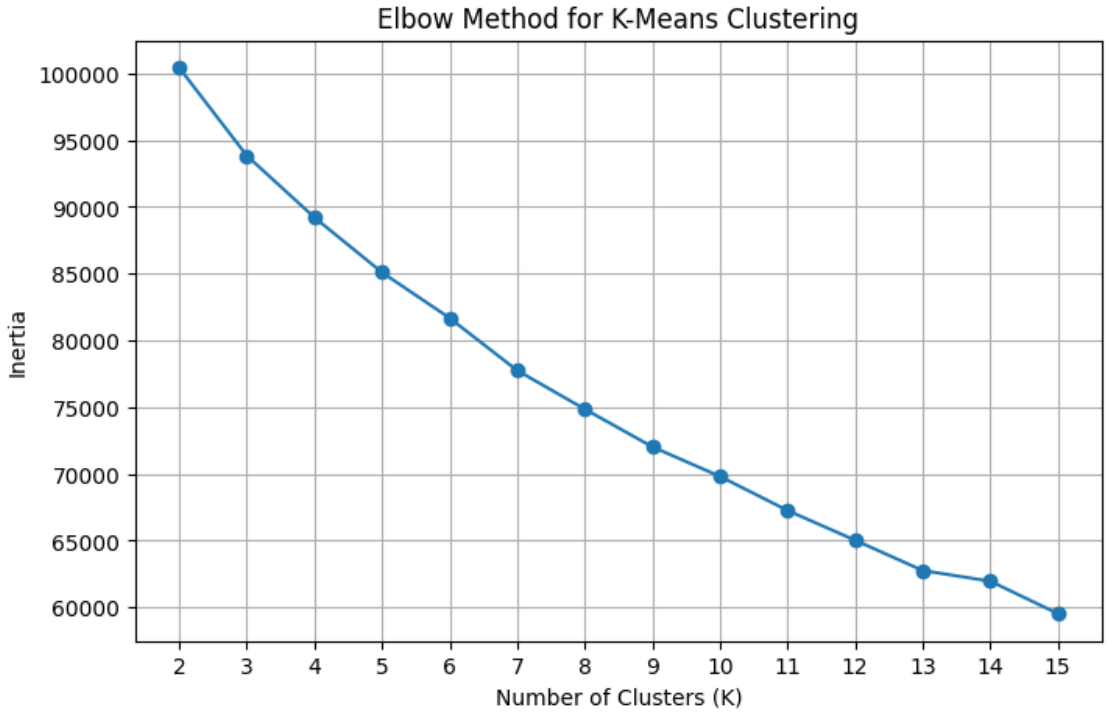
[ ]:
```
plt.figure(figsize=(8, 5))

plt.plot(k_values, inertia, marker='o')

plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method for K-Means Clustering")

plt.xticks(k_values)
plt.grid(True)

plt.show()
```




[ ]:
```
plt.figure(figsize=(8, 5))

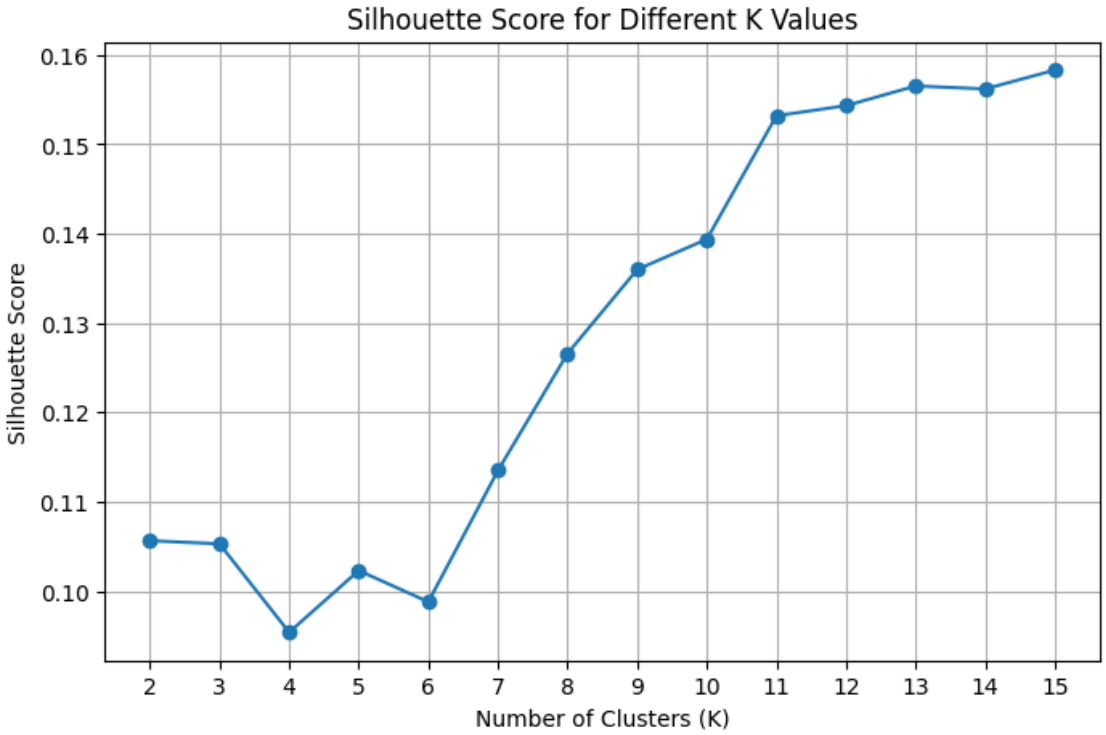
plt.plot(k_values, silhouette_scores, marker='o')

plt.xlabel("Number of Clusters (K)")
```

```
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score for Different K Values")

plt.xticks(k_values)
plt.grid(True)

plt.show()
```




[ ]:
```
best_index = np.argmax(silhouette_scores)

best_k = list(k_values)[best_index]
best_score = silhouette_scores[best_index]

print("Best K:", best_k)
print("Highest Silhouette Score:", best_score)
```

```
Best K: 15
Highest Silhouette Score: 0.15830708064242807
```

[ ]:
```
kmeans_final = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
```

```
)

cluster_labels = kmeans_final.fit_predict(X_scaled)

print("Cluster labels:")
print(cluster_labels[:20])
```

```
Cluster labels:
[ 6  1  1  3 10  3  5  4  3  3  6  2  9  3 10  7  5  4  1  3]
```

[ ]:
```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)

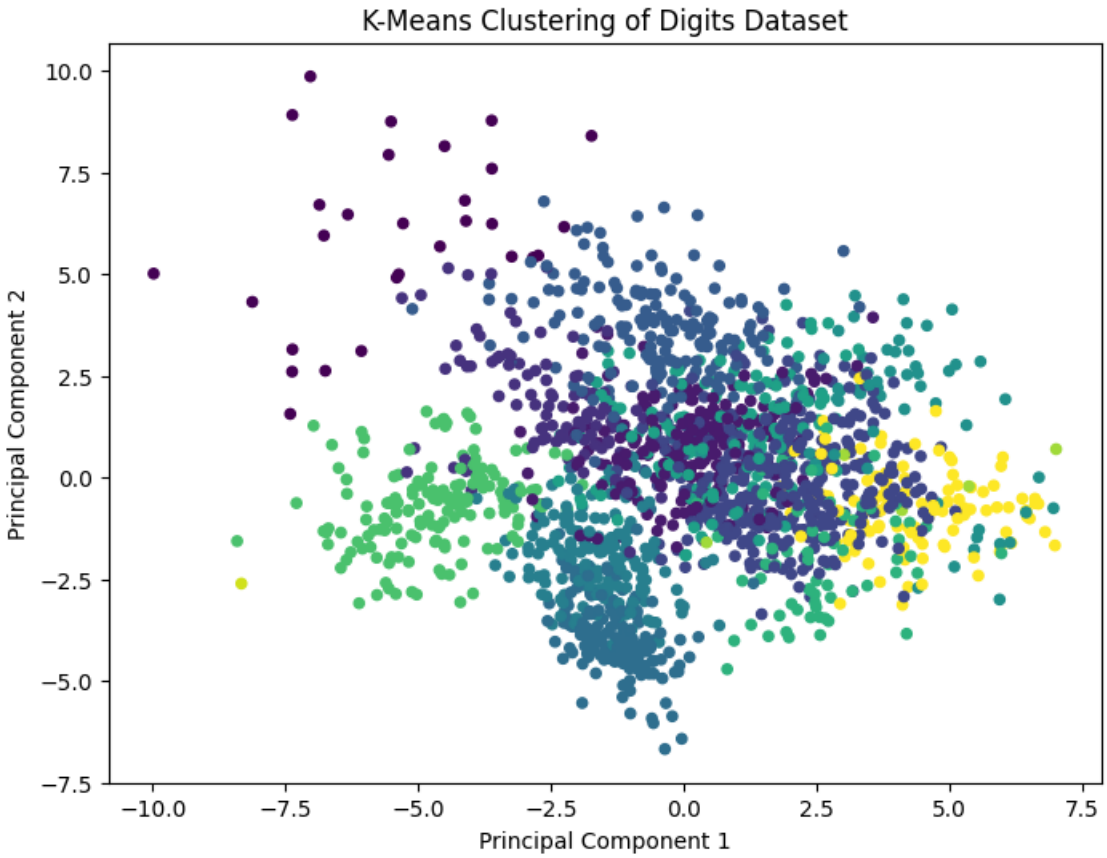
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=cluster_labels,
    s=20
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clustering of Digits Dataset")

plt.show()
```


K-Means Clustering of Digits Dataset
Principal Component 2
Principal Component 1
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5